FLASK

Framework Web Python

untuk Deployment Machine Learning

Ringan | Fleksibel | Production-Ready

...

```
@app.route('/')
```

Contoh Implementasi: Titanic · HeartCheck · Regresi · ARIMA

Apa itu Flask?

Definisi

Flask adalah micro web framework berbasis Python yang dikembangkan oleh Armin Ronacher pada tahun 2010. Flask bersifat minimalis — tidak memaksakan struktur aplikasi tertentu dan tidak menyertakan komponen bawaan seperti database layer atau form validation.

"Micro" bukan berarti kecil — melainkan sederhana dan dapat diperluas sesuai kebutuhan.

Komponen Inti

- Werkzeug (WSGI toolkit)
- Jinja2 (template engine)
- Click (CLI)

Bahasa

Python 3.x

Tipe

Micro Framework

Ukuran

< 100KB

GitHub Stars

68K+

Lisensi

BSD-3-Clause

Versi Stabil

Flask 3.x

Fungsi & Kegunaan Flask

Web Server & Routing

Mendaftarkan URL endpoint dan menangani request HTTP (GET, POST, PUT, DELETE). Setiap route dihubungkan dengan fungsi Python via dekorator @app.route.

Rendering Template

Menggunakan Jinja2 untuk menghasilkan halaman HTML dinamis. Variabel dari Python dikirim ke HTML dan di-render secara server-side.

Serving Model ML

Memuat model machine learning (joblib, pickle, ONNX) dan menyediakan REST API endpoint sehingga model bisa diakses via browser atau aplikasi lain.

Manajemen Data

Menerima input form HTML, memvalidasi data, lalu meneruskan ke model atau database. Mendukung SQLite, PostgreSQL, dan ORM seperti SQLAlchemy.

Autentikasi & Sesi

Mengelola session pengguna, cookie, dan login/logout. Flask-Login dan Werkzeug mempermudah implementasi sistem autentikasi yang aman.

REST API Backend

Membangun API endpoint yang mengembalikan JSON. Cocok sebagai backend untuk aplikasi React, Vue, Flutter, atau mobile app.

Kenapa Flask?

Alasan Flask menjadi pilihan utama untuk deployment model ML

#1
Micro Framework
Python Terpopuler

68K+
GitHub Stars
(dan terus bertambah)

<1min
Setup awal
hingga berjalan

100%
Bebas pilih
komponen sendiri

Mudah dipelajari

Syntax intuitif, dokumentasi lengkap. Hello World bisa dibuat dalam 5 baris kode.

Fleksibel & modular

Tidak ada struktur paksa — tambahkan hanya komponen yang benar-benar dibutuhkan.

Ideal untuk prototyping

Cepatkan proses dari eksperimen Jupyter Notebook ke aplikasi web yang bisa diakses tim.

Siap production

Digunakan Netflix, LinkedIn, Reddit, Airbnb. Terbukti bisa menangani jutaan request.

Integrasi ML sempurna

Load model sklearn/joblib, terima input JSON, kembalikan prediksi — hanya butuh beberapa baris.

Ekosistem luas

Flask-Login, Flask-SQLAlchemy, Flask-RESTful, dll. Memperluas kemampuan sesuai kebutuhan.

Flask vs Framework Python Lainnya

Fitur	Flask	Django	FastAPI
Tipe	Micro Framework	Full-stack Framework	Modern ASGI Framework
Kurva Belajar	⭐ Mudah	⭐⭐⭐ Menengah	⭐⭐ Ringan
ORM Bawaan	✗ Tidak ada	✓ Django ORM	✗ Tidak ada
Admin Panel	✗ Tidak ada	✓ Built-in	✗ Tidak ada
Async Support	Terbatas	Terbatas	✓ Native async
Performa	Baik	Baik	Sangat Baik
Template Engine	Jinja2	DTL / Jinja2	Opsional
Cocok untuk ML	✓ Sangat cocok	Bisa, lebih berat	✓ Sangat cocok
Ukuran Project	Kecil–Menengah	Menengah–Besar	API-first

Flask vs Gradio — Perbandingan Mendalam

🔵 FLASK	VS	🟠 GRADIO
General-purpose web framework — untuk semua jenis aplikasi web	**Tujuan**	Khusus demo model ML — buat UI cepat untuk model AI/ML
Sepenuhnya custom — HTML, CSS, JS bebas didesain sesuai kebutuhan	**Desain UI**	Komponen pre-built (slider, image, text) — tampilan terbatas
Full control atas routing, template, logic, database, autentikasi	**Kontrol**	Sangat terbatas — hanya untuk input/output model sederhana
✅ Production-ready — bisa deploy ke server, cloud, container	**Production**	⚠️ Lebih cocok untuk demo & prototyping cepat (Hugging Face Spaces)
Perlu belajar routing, HTML, Jinja2 — investasi waktu awal lebih tinggi	**Kurva Belajar**	Sangat cepat — 3-5 baris kode sudah dapat UI yang berjalan

💡 Kesimpulan: Gunakan Gradio untuk demo cepat di lab/notebook. Gunakan Flask ketika butuh aplikasi yang dapat dikustomisasi, production-ready, dan memiliki UX yang baik.

Cara Kerja Flask — Request & Response Cycle

1 Browser / Client
User buka URL

2 WSGI Server
Werkzeug / Gunicorn

3 Flask Router
@app.route('/')

4 View Function
Logic & Model ML

5 Response
HTML / JSON

app.py

```python
from flask import Flask, render_template, request
import joblib, numpy as np

app = Flask(__name__)
model = joblib.load('model.joblib')

@app.route('/', methods=['GET', 'POST'])
def index():
    result = model.predict(features)
    return render_template('index.html')
```

`app = Flask(__name__)`
Inisialisasi aplikasi Flask. __name__ memberitahu Flask lokasi folder templates & static.

`@app.route('/path')`
Dekorator untuk mendaftarkan fungsi sebagai handler URL tertentu.

`request.method`
Cek apakah request adalah GET (buka halaman) atau POST (submit form).

`render_template()`
Render file HTML dari folder /templates/ dengan variabel dinamis Jinja2.

`app.run(debug=True)`
Jalankan server development. Debug mode: auto-reload saat kode berubah.

Template Jinja2 — HTML Dinamis dengan Python

templates/index.html

```
<!-- Variabel dari Flask -->
<h2>Hasil: {{ prediction }}</h2>
<p>Akurasi: {{ accuracy }}%</p>

<!-- Kondisional if/else -->
{% if prediction == 'Selamat' %}
  <div class='success'>✓ Selamat!</div>
{% else %}
  <div class='danger'>✗ Tidak</div>
{% endif %}

<!-- Loop untuk riwayat -->
{% for item in riwayat %}
  <li>{{ item.date }} - {{ item.result }}</li>
{% endfor %}
```

{{ }}

Variabel

Sisipkan variabel Python ke HTML. Contoh: {{ prediction }} akan diganti nilai dari Flask.

{% %}

Statement

Kontrol alur: if/else, for loop, block, extends. Membuat HTML yang dinamis dan reusable.

{# #}

Komentar

Komentar dalam template yang tidak ditampilkan di output HTML akhir.

|filter

Filter

Transformasi data: {{ value|round(2) }}, {{ text|upper }}, {{ list|length }}.

💡 Tips: Gunakan {% extends 'base.html' %} untuk menghindari duplikasi kode di

Integrasi Flask dengan Model Machine Learning

Dari notebook eksperimen ke aplikasi web yang bisa diakses semua orang

1 Train Model
Jupyter Notebook
sklearn, joblib

→

2 Save Model
joblib.dump()
model.joblib

→

3 Load di Flask
joblib.load()
pada startup

→

4 API Endpoint
@app.route()
terima input

→

5 Prediksi
model.predict()
kembalikan hasil

Integrasi Lengkap

```python
# Load model sekali saat startup
model = joblib.load('model.joblib')

@app.route('/predict', methods=['POST'])
def predict():
    age  = float(request.form['age'])
    X    = np.array([[age, ...]])
    pred = model.predict(X)[0]
    prob = model.predict_proba(X)[0]
```

Load sekali, prediksi jutaan kali
Model di-load saat server start. Tidak perlu reload tiap request — sangat efisien.

Confidence Score
predict_proba() memberikan probabilitas per kelas, bukan sekedar label.

Input Validation
Selalu validasi input sebelum dimasukkan ke model — try/except untuk handling error.

Pipeline Preprocessing
Gunakan sklearn Pipeline agar preprocessing di training = di inference.

Demo 1 — Titanic Survival Predictor (Versi Sederhana)

🚢 Titanic Survival Predictor

Tujuan Pembelajaran:

Memahami alur dasar Flask — dari input form HTML hingga menampilkan hasil prediksi model machine learning.

Dataset	Titanic (200 baris inline)
Model	Decision Tree (max_depth=4)
Fitur Input	Pclass, Sex, Age, Fare
Output	Selamat / Tidak Selamat
Tambahan	Confidence score + akurasi model
Port	http://127.0.0.1:5000

localhost:5000

🚢 Titanic Survival Predictor

Kelas Penumpang

2 (Kelas 2)

Jenis Kelamin

Female ▼

Usia (tahun)

25

Harga Tiket (fare)

50.0

🔮 Prediksi Sekarang

📊 Info Model

Akurasi: 82.5%

Model: Decision Tree

Max Depth: 4

✅ **SELAMAT**

Tingkat kepercayaan: 78.5%

Demo 2 — HeartCheck: Heart Disease Predictor

💚 Tema Kesehatan Hijau-Putih · 918 pasien · Random Forest + Pipeline · Autentikasi SQLite

🏠 Landing Page

Hero page profesional dengan statistik model (akurasi, AUC-ROC), cara kerja sistem, dan CTA.

📋 Form Prediksi

11 fitur klinis dengan tooltip ⓘ, 6 profil contoh pasien realistis, hasil prediksi di bawah form.

🔐 Autentikasi

Sistem daftar & masuk akun menggunakan SQLite + Werkzeug bcrypt hash. Session management Flask.

📜 Riwayat Prediksi

Setiap prediksi disimpan ke database SQLite jika pengguna login. Ditampilkan sebagai accordion card.

📊 Feature Importance

Bar chart top-10 fitur paling penting dalam model. Menjelaskan keputusan model secara visual.

ℹ️ Halaman Tentang

Konteks medis, pipeline model, FAQ, dan disclaimer. Transparansi untuk pengguna awam.

Random Forest · Pipeline + StandardScaler · OneHotEncoder · Cross-Validation · AUC-ROC · SQLite + Werkzeug · Chart.js

Demo 3 & 4 — Regresi dan Time Series ARIMA

🏠 Regresi — House Price Estimator

Dataset	Boston Housing (195 baris inline)
Model	Gradient Boosting Regressor
Evaluasi	R^2 Score + Mean Absolute Error
Fitur Input	RM, LSTAT, PTRATIO, CHAS, RAD
Output	Estimasi Harga + Interval ±MAE
Port	http://127.0.0.1:5002

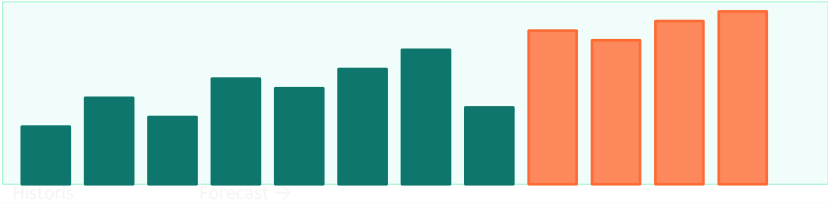
R^2 Score: 0.85

- Interval kepercayaan ±MAE
- Gauge bar posisi harga
- Landing page penjelasan model

- Interval kepercayaan ±MAE
- Gauge bar posisi harga relatif
- Landing page + penjelasan model

📈 ARIMA — Whoosh Passenger Forecast

Dataset	Sintetis penumpang Whoosh (30 bulan)
Model	ARIMA(1,1,1) — statsmodels
Rute	Jakarta–Bandung (Kereta Cepat)
Horizon	Pilih 3–24 bulan ke depan
Visualisasi	Chart.js: historis + garis forecast
Port	http://127.0.0.1:5003



Struktur Folder Proyek Flask

Berdasarkan contoh proyek di folder ini

```
• • •   contoh-flask/

contoh-flask/
├── app.py                ← Entry point Flask
├── train_simple.py       ← Training model
├── model.joblib          ← Model tersimpan
├── requirements.txt      ← Dependensi
├── templates/            ← HTML Jinja2
│   └── index.html
├── klasifikasi/          ← HeartCheck App
│   ├── app.py
│   ├── database.py       ← SQLite helper
│   ├── model.joblib
│   └── templates/
│       ├── base.html     ← Master layout
│       ├── prediksi.html
│       └── riwayat.html
├── regresi/              ← House Price App
└── arima/                ← Whoosh Forecast
```

app.py sebagai entry point

Semua route, konfigurasi Flask, dan inisialisasi model ada di sini. Load model sekali di luar fungsi route.

Folder templates/ untuk HTML

Flask mencari template di folder ini secara otomatis. Gunakan base.html + extends untuk DRY (Don't Repeat Yourself).

model.joblib di root

Simpan model terlatih dengan joblib. Pisahkan script training (train.py) dari script serving (app.py).

requirements.txt

Selalu catat semua dependensi. Jalankan pip freeze > requirements.txt setelah semua library terinstall.

venv untuk isolasi

Gunakan virtual environment agar tidak konflik dengan proyek Python lain di komputer yang sama.

Cara Memulai & Menjalankan Proyek Flask

01 Install & Buat Virtual Environment

```
python -m venv venv
venv\Scripts\activate   # Windows
source venv/bin/activate  # Mac/Linux
```

02 Install Dependensi

```
pip install -r requirements.txt
# Atau manual:
pip install flask scikit-learn joblib numpy
```

03 Training Model (sekali saja)

```
python train_simple.py
# Output: model.joblib tersimpan
# Berisi: model + accuracy score
```

04 Jalankan Server Flask

```
python app.py
# Output: * Running on http://127.0.0.1:5000
# Buka URL di browser!
```

💡 Pro Tips: debug=True untuk development (auto-reload kode). Nonaktifkan di production! Gunakan Gunicorn untuk deployment server sesungguhnya.

KESIMPULAN

Flask:

Framework Pilihan untuk ML Deployment

- Ringan, fleksibel, mudah dipelajari
- Ideal untuk prototyping & production
- Integrasi mulus dengan sklearn & joblib
- Kontrol penuh atas UI dan logika

Mulai Sederhana

Titanik Predictor hanya butuh ~50 baris kode. Cukup untuk memahami flow lengkap Flask → Model → UI.

Iterasi Cepat

Dari Jupyter Notebook ke web app dalam hitungan jam. debug=True membantu pengembangan yang cepat.

Skalabel

Dimulai dari micro app, bisa berkembang ke full-stack dengan autentikasi, database, dan API lengkap.

Komunitas Besar

Dokumentasi lengkap di flask.palletsprojects.com. Ribuan tutorial dan library ekstensi siap pakai.